

Ex.no:9	MINI PROJECT – FINANCIAL PERFORMANCE ANALYTICS

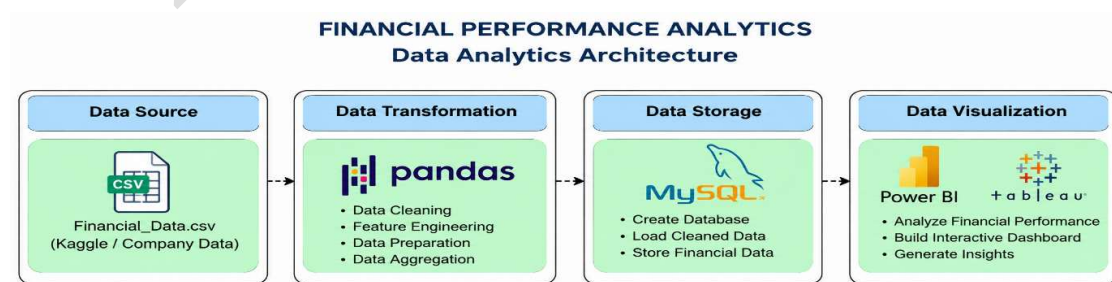
Aim:

To clean, transform, store, and analyze S&P 500 financial statement data using Python, MySQL, PowerBI and Tableau to study revenue, profitability, operating performance, expenses, and financial relationships.

Algorithm:

1. Load the S&P 500 financial dataset from the CSV file using Pandas.
2. Inspect the dataset for missing values, duplicate records, data types, and inconsistent date formats.
3. Remove the unnecessary index column and records with missing critical financial statement values.
4. Convert financial fields to numeric format and handle remaining missing numerical values using median imputation.
5. Convert the date field into datetime format and create year and quarter attributes for time-based analysis.
6. Create derived attributes such as profit_margin, operating_margin, gross_margin, rd_expense_ratio and operating_expense_ratio.
7. Standardize column names and perform a final data quality check for missing values and duplicate records.
8. Store the cleaned and transformed financial dataset in a MySQL database named financial_performance.
9. Create SQL queries to analyze company-wise revenue, net income, profitability, quarterly performance, and expense ratios.
10. Connect the MySQL database to Power BI and create DAX measures for revenue, net income, gross profit, operating income, companies, and profitability.
11. Create complementary Tableau visualizations for expense structure, financial relationships, R&D investment, and expense composition by year.
12. Generate meaningful financial insights from the dashboards.

Architecture:



Program:

1. Python – Data Cleaning & Transformation

```
import pandas as pd
# Load the financial dataset
df = pd.read_csv("financial data sp500 companies.csv")
financial = df.copy()
# Check the original dataset
print("Original Shape:", financial.shape)
print("\nMissing Values Before Cleaning:")
missing_before = financial.isnull().sum()
print(missing_before[missing_before > 0])
print("\nDuplicate Records Before Cleaning:", financial.duplicated().sum())
# Remove unnecessary index column
financial.drop(columns=["Unnamed: 0"], inplace=True)
# Remove duplicate records
financial.drop_duplicates(inplace=True)
# Convert date to datetime
financial["date"] = pd.to_datetime(financial["date"], errors="coerce")
# Remove records with missing critical financial values
critical_columns = [
    "Income Before Tax",
    "Operating Income",
    "Net Income From Continuing Ops",
    "Net Income Applicable To Common Shares"
]
financial.dropna(subset=critical_columns, inplace=True)
# Convert financial columns to numeric
numeric_columns = financial.select_dtypes(include="number").columns
financial[numeric_columns] = financial[numeric_columns].apply(
    pd.to_numeric, errors="coerce"
)
# Fill remaining missing numerical values with column median
for column in numeric_columns:
    financial[column] = financial[column].fillna(financial[column].median())
# Create year and quarter attributes
financial["year"] = financial["date"].dt.year
financial["quarter"] = financial["date"].dt.quarter
# Create financial ratios
financial["profit_margin"] = (
    financial["Net Income"] / financial["Total Revenue"]
) * 100
financial["operating_margin"] = (
    financial["Operating Income"] / financial["Total Revenue"]
) * 100
financial["gross_margin"] = (
    financial["Gross Profit"] / financial["Total Revenue"]
) * 100
financial["rd_expense_ratio"] = (
    financial["Research Development"] / financial["Total Revenue"]
) * 100
financial["operating_expense_ratio"] = (
    financial["Total Operating Expenses"] / financial["Total Revenue"]
) * 100
# Replace infinite values created by division by zero
financial.replace([float("inf"), float("-inf")], pd.NA, inplace=True)
# Standardize column names
financial.columns = (
    financial.columns
    .str.lower()
    .str.strip()
    .str.replace(" ", "_", regex=False)
)
```

```

# Final data quality check
print("\nMissing Values After Cleaning:")
missing_after = financial.isnull().sum()
print(missing_after[missing_after > 0])
print("\nDuplicate Records After Cleaning:", financial.duplicated().sum())
print("\nFinal Shape:", financial.shape)
print("\nData Types After Cleaning:")
print(financial.dtypes)
print("\nFirst 5 Cleaned Records:")
display(financial.head())
# The cleaned financial DataFrame is now ready for storage and analysis.
print("Rows ready for MySQL:", len(financial))

```

Python – Output:

Original dataset shape: 2012 rows × 19 columns.

After removing the unnecessary index column and one record with missing critical financial values, the cleaned dataset contains 2011 rows × 25 columns.

The cleaned dataset contains 502 companies and 502 unique tickers, with year/quarter attributes and five derived financial ratios.

```

Final Dataset Size: 2008 records and 25 columns

Data Types After Transformation:
date                                datetime64[us]
firm                                str
ticker                              str
research_development                float64
income_before_tax                   float64
net_income                          float64
selling_general_administrative       float64
gross_profit                        float64
ebit                                float64
operating_income                    float64
interest_expense                    float64
income_tax_expense                  float64
total_revenue                       float64
total_operating_expenses             float64
cost_of_revenue                     float64
total_other_income_expense_net       float64
net_income_from_continuing_ops       float64
net_income_applicable_to_common_shares float64
profit_margin                        object
operating_margin                     object
gross_margin                         object
rd_expense_ratio                     object
operating_expense_ratio              object
year                                int32
quarter                              str
dtype: object

Original Dataset: 2012 records and 19 columns

Columns With Missing Values Before Cleaning:
Research Development      1378
Income Before Tax         1
Selling General Administrative  60
Operating Income          1
Interest Expense          182
Net Income From Continuing Ops  1
Net Income Applicable To Common Shares  1
dtype: int64

Duplicate Records Found: 0

Columns With Missing Values After Cleaning:
No missing values remain after cleaning.

Duplicate Records After Cleaning: 0

```

2. MySQL – Database and Data Loading

MySQL:

```
CREATE DATABASE IF NOT EXISTS financial_performance;  
USE financial_performance;
```

Python/Jupyter:

```
from sqlalchemy import create_engine  
  
username = "root"  
password = "Dinesh2214"  
host = "localhost"  
port = 3306  
  
engine = create_engine(  
    f"mysql+pymysql://{username}:{password}"  
    f"@{host}:{port}/financial_performance"  
)  
  
print("MySQL connection created successfully!")  
  
financial.to_sql(  
    name="financial_data",  
    con=engine,  
    if_exists="replace",  
    index=False  
)  
  
print("Cleaned financial data loaded into MySQL successfully.")  
print("Number of rows loaded:", len(financial))
```

Verify:

```
pd.read_sql(  
    "SELECT COUNT(*) AS total_rows FROM player_performance",  
    db_engine  
)
```

3. MySQL – Analysis Queries

Total records

```
SELECT COUNT(*) AS total_records  
FROM financial_data;
```

	total_records
▶	2008

2. Company-wise financial performance

```
SELECT  
    firm,  
    ROUND(SUM(total_revenue) / 1000000000, 2) AS revenue_billion,
```

```

ROUND(SUM(net_income) / 1000000000, 2) AS net_income_billion,
ROUND(AVG(profit_margin), 2) AS avg_profit_margin
FROM financial_data
GROUP BY firm
ORDER BY net_income_billion DESC;

```

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
	firm	revenue_billion	net_income_billion	avg_profit_margin
▶	Apple	347.16	86.8	24.62
	Alphabet (Class A)	220.26	62.93	28.37
	Alphabet (Class C)	220.26	62.93	28.37
	Microsoft	168.09	61.27	36.5
	JPMorgan Chase	128.84	47.83	36.96
	Facebook	104.79	38.96	37.14
	Bank of America	91.2	30.44	33.18
	Amazon	443.3	29.44	6.67
	Citigroup	71.46	21.34	29.08
	Goldman Sachs	55.2	20.2	36.33
	Verizon Communic...	132.87	19.99	15.04
	...	...	...	...

3. Top 10 companies by profitability

```

SELECT
    firm,
    ROUND(AVG(profit_margin), 2) AS avg_profit_margin,
    ROUND(AVG(operating_margin), 2) AS avg_operating_margin,
    ROUND(AVG(gross_margin), 2) AS avg_gross_margin
FROM financial_data
GROUP BY firm
ORDER BY avg_profit_margin DESC
LIMIT 10;

```

firm	avg_profit_margin	avg_operating_margin	avg_gross_margin
Bio-Rad Laboratories	142.88	19.86	57.36
eBay	120.39	27.58	75.2
The Cooper Companies	104.42	22.43	69.24
Regeneron Pharmaceuticals	50.77	50.56	62.39
Public Storage	50.39	50.93	73.28
Verisign	48.56	64.63	85.61
AvalonBay Communities	48.37	27.01	62.07
Visa	47.92	64.68	96.76
Alexandria Real Estate Equities	47.51	26.2	70.99
Extra Space Storage	43.6	50.93	75.36

4. Year-wise financial performance

```

SELECT
    year,
    ROUND(SUM(total_revenue) / 1000000000, 2) AS revenue_billion,
    ROUND(SUM(net_income) / 1000000000, 2) AS net_income_billion,
    ROUND(AVG(profit_margin), 2) AS avg_profit_margin

```

FROM financial_data
 GROUP BY year
 ORDER BY year;

Result Grid				
		Filter Rows:	Export:	Wrap Cell Content:
	year	revenue_billion	net_income_billion	avg_profit_margin
▶	2020	4263.07	378.52	-50.67
	2021	8874.94	1057.38	-34.05

5. Quarterly financial performance

```
SELECT
  quarter,
  ROUND(SUM(total_revenue) / 1000000000, 2) AS revenue_billion,
  ROUND(SUM(net_income) / 1000000000, 2) AS net_income_billion,
  ROUND(AVG(profit_margin), 2) AS avg_profit_margin
FROM financial_data
GROUP BY quarter
ORDER BY quarter;
```

Result Grid				
		Filter Rows:	Export:	Wrap Cell Content:
	quarter	revenue_billion	net_income_billion	avg_profit_margin
▶	2020Q1	0.25	0.04	14.61
	2020Q2	184.75	6.07	-124.96
	2020Q3	1725.82	171.69	-60.91
	2020Q4	2352.25	200.72	-31.53
	2021Q1	5976.88	653.28	-49.62
	2021Q2	2283.39	328.98	3.18
	2021Q3	614.68	75.11	13.88

3. Power BI – DAX Measures

Total Revenue

Total Revenue = SUM(financial_data[total_revenue])

Total Net Income

Total Net Income = SUM(financial_data[net_income])

Total Gross Profit

Total Gross Profit = SUM(financial_data[gross_profit])

Total Operating Income

Total Operating Income = SUM(financial_data[operating_income])

Total Companies

Total Companies = DISTINCTCOUNT(financial_data[ticker])

Average Profit Margin

Average Profit Margin = AVERAGE(financial_data[profit_margin])

Average Operating Margin

Average Operating Margin = AVERAGE(financial_data[operating_margin])

Power BI visuals created include KPI cards, top companies by revenue, top companies by net income, revenue and net income trend, profitability comparison, and revenue versus net income analysis.

4. Tableau Dashboard

The cleaned financial CSV file was connected to Tableau Public and complementary worksheets were created to explore expense structure, financial relationships, R&D investment, and expense composition.

Output:

POWERBI DASHBOARD:

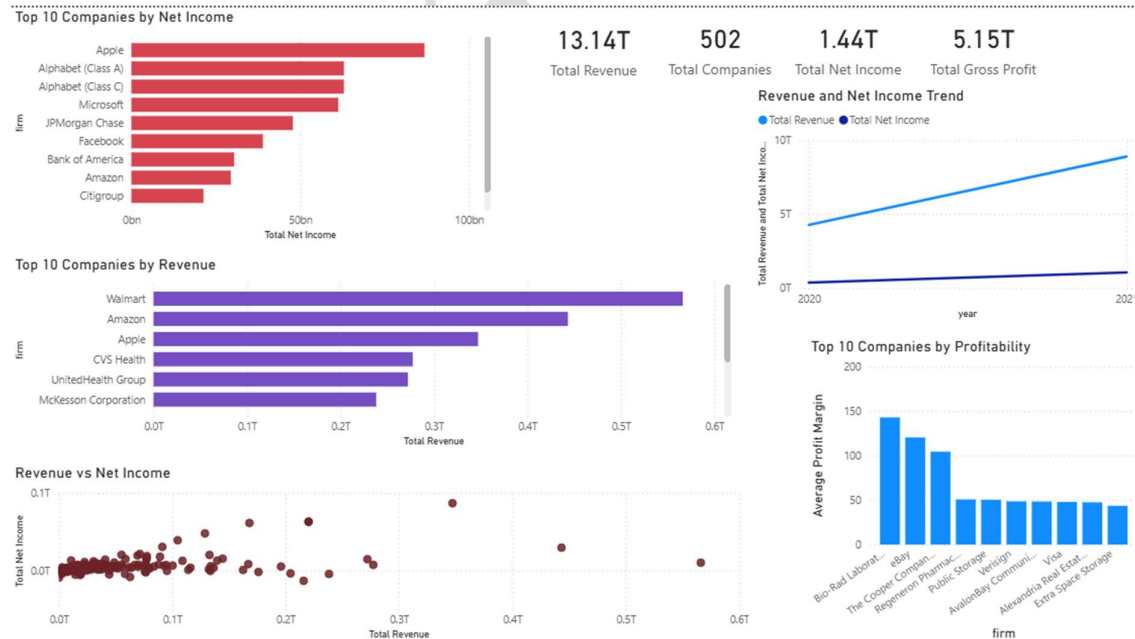
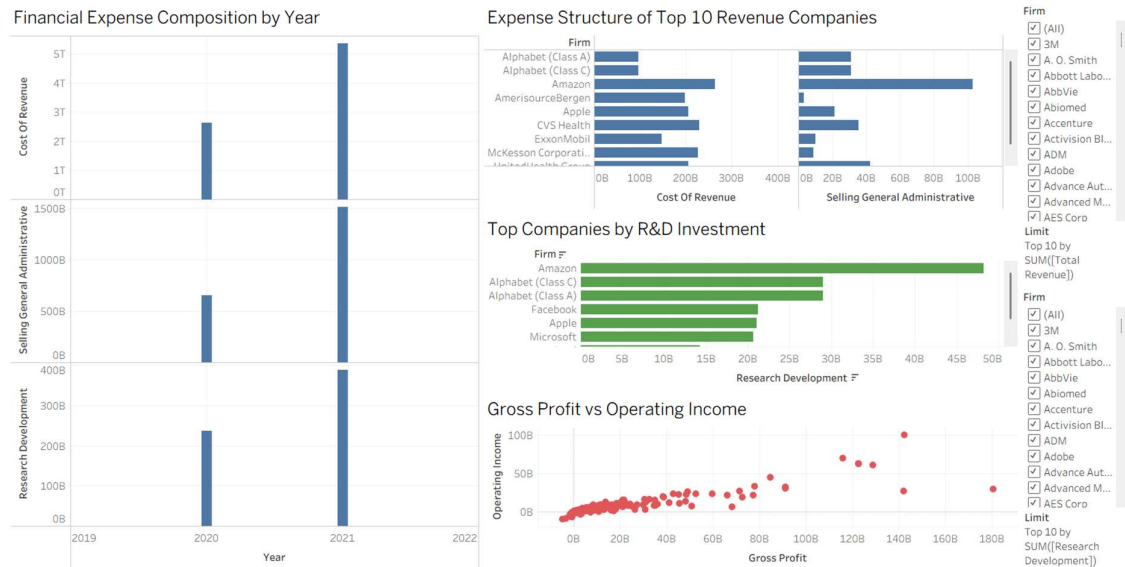


TABLEAU DASHBOARD:



Result: